pdf-to-markdown

Release notes, version 2.3

11 October 2026

Summary

`pdf-to-markdown` is a Claude skill that converts PDFs of physics papers, notes and textbooks into Markdown with LaTeX mathematics. Every formula is transcribed from page images rather than from the PDF's text layer, then checked mechanically and proofread against the original.

Version 2.3 adds figures. Each figure is found from its caption, rendered whole by a script, and embedded in the Markdown as image data, so the result is one self-contained `.md` file. Claude writes only a placeholder and the caption, never the image data, so figures add almost no tokens. On an eight-page, two-column PNAS paper all seven figures were found, cropped completely and embedded byte for byte.

Version 2.2 made the skill cheaper to run without changing how formulas are read: page strips are rendered at 170 dpi instead of 300 dpi, the text layer is no longer read page by page, every token of output is written once, and the preparation script prints everything the transcription needs on one line per page. On the five-page test note this is estimated to cut a run from about 45,000 to about 30,000 tokens. The lower resolution was adopted only after a blind legibility test and an open blind transcription of two unseen pages, both without a single error (Section 5).

The transcription evidence still comes from born-digital, single-column documents only; figure extraction has been tested on one two-column journal paper. The known limitations in Section 7, in particular scanned PDFs and two-column layouts, should be read before relying on the skill elsewhere.

Version history

Version	Change
1.0	Draft only, never packaged. markitdown converted each page; pages whose text contained many mathematical Unicode characters were re-transcribed by Claude from 200 dpi page images.
2.0	First packaged release. markitdown removed for PDFs; math detected by font; strips cut between text lines; mechanical checks (`check.py`) and side-by-side proofreading (`verify_eqs.py`) added; evaluation scripts added.
2.1	Scorer (`scripts/eval/mathcmp.py`) rewritten: content-based alignment, tolerance for spelling-only variants, `--lenient` option, exact-match counts. No change to conversion.
2.2	Token efficiency: strips at 170 dpi; no page-by-page reading of the text layer; per-page output files joined by the new `assemble.py`; compact `prepare.py` printout. No change to the transcription rules.
2.3	Figures: new `figures.py` finds figures by caption and renders them; `assemble.py` embeds them as base64 data, giving one self-contained file. Text and math handling unchanged.

1 Why version 1.0 was replaced

Version 1.0 was run on the test document and compared with its LaTeX source. The text layer, through markitdown or otherwise, proved unusable for mathematics:

- **Invented tables.** markitdown 0.1.5 split ordinary sentences on page 2 into 60 rows of Markdown table cells.
- **Lost typefaces.** Calligraphic $\mathcal{G}$ and roman G both came out as the same letter G, and bold $\boldsymbol{\omega}$ as a plain ω. The test note's notation rests on exactly this distinction.
- **Detached accents.** $\tilde{\mathrm{K}} = 1/\Omega_{\bar{1}}$ was extracted as `K = 1 ~ /Ω¯1`, the tilde moved off the K and next to the 1.
- **Placeholder glyphs.** Integral and sum signs and brace pieces leaked as `(cid:90)`, `(cid:88)` and similar codes.
- **Useless page flagging.** The Unicode-count rule flagged all five pages, since every page of a physics note contains mathematics.
- **Broken paragraphs.** Page markers landed in the middle of sentences that continue across a page break.

2 What version 2 does

File	Role
`SKILL.md`	Instructions to Claude: ground rules, the six-step procedure, writing conventions, context budget.
`scripts/prepare.py`	Per page: 170 dpi strips cut in the gaps between lines, a text layer for searching single words, the share of characters set in math fonts, and the equation numbers printed at the right margin. Prints one line per page: image or text, each strip with its y-range in points, tags.
`scripts/figures.py`	Finds each figure from its caption, renders the whole region at 200 dpi, and writes a low-resolution check image with the crop outlined in red.
`scripts/assemble.py`	Joins the per-page Markdown files in page order; rejoins a paragraph that a page break interrupted; replaces figure placeholders with the embedded images.
`scripts/check.py`	Every formula must render in KaTeX; every printed equation number must appear exactly once as `\tag{}`; no text-layer debris.
`scripts/verify_eqs.py`	Retypesets each numbered display equation with pdflatex and places it under the PDF's original, four equations per sheet.
`scripts/eval/mathcmp.py`	Formula-by-formula comparison with a LaTeX source; works on any tool's Markdown (testing only).
`scripts/eval/prosecmp.py`	Word-by-word prose comparison with a LaTeX source (testing only).

Writing conventions enforced by `SKILL.md`: typefaces reproduced exactly ($\mathcal{K}$, K, $\boldsymbol{\Omega}$, upright i, d, e where printed so); printed equation numbers used in `\tag{}`, even when out of sequence; zoomed crops at 500–600 dpi before deciding any multi-character bar such as $\overline{j+1}$ versus $\bar{j}+1$, any tilde, hat or prime, and the extent of every brace; each figure as a placeholder `![Fig. N](figure:figN)` followed by its caption, transcribed like any other text.

3 What changed in 2.3

- **Figure detection by caption.** `figures.py` looks for text blocks starting "Fig. N", "Figure N" or "FIG. N". The figure is everything graphical directly above the caption and within its column (vector paths, embedded images and the small text inside them, such as axis labels), up to the nearest line of running prose. Decorative graphics without a caption, such as journal badges and ORCID icons, are ignored automatically.
- **Rendering instead of extracting.** Physics figures are usually composites: vector line art with small embedded raster pieces such as heatmaps and colorbars. Extracting the embedded images would yield fragments, so the whole region is rendered at 200 dpi. A PNG larger than 1.5 MB is saved as JPEG instead.
- **Crop checks.** For every figure a low-resolution view of its surroundings is written with the crop drawn as a red box; Claude confirms that nothing is cut off. Several check images can be combined into one view.
- **Embedding by script.** In the page Markdown Claude writes `![Fig. N](figure:figN)` and the caption. `assemble.py` replaces each placeholder with a base64 data URI and stops with an error if an image is missing. Writing the data by hand would cost tens of thousands of output tokens per figure; this way the cost is a few hundred tokens of crop checking per figure.
- **Instructions.** `SKILL.md` gains the figure step, the placeholder convention, a rule never to view the assembled file whole (the image data is large) and a note on viewer support.

4 What changed in 2.2

- **Strips at 170 dpi.** `prepare.py` now renders strips at 170 dpi by default. At 300 dpi a strip was about 2,000 px wide and was downscaled by the model to an effective 232 dpi anyway; at 170 dpi it is about 1,100 px wide and is seen as rendered. Strip tokens fall by about 46%.
- **No reading of the text layer.** `SKILL.md` forbids reading `page_NNN.txt` as a whole; it may only be searched for one doubtful word. Prose read from the strips was error-free in every test.
- **Every token of output written once.** Each page is written with `create_file` to its own `page_NNN.md`. A page ending mid-paragraph ends with the line `<!-- continues -->`. `assemble.py` joins the pages and rejoins such paragraphs; corrections after checking and proofreading are targeted `str_replace` edits. The document is never re-typed.
- **Compact script output.** `prepare.py` prints one line per page with the strip files, their y-ranges in points (for zooms) and the equation tags, so `manifest.json` never needs to be opened.
- **Proofreading printout.** Step 5 now requires reading the full output of `verify_eqs.py` first: a `PROBLEM` line marks an equation that could not be typeset and is therefore missing from the sheets. This is an instruction only; the script itself is unchanged.
- **Documentation.** `--lenient`, exact-count reading of scores and scoring other tools' output are now described in `SKILL.md`.

Estimated tokens for one run on the five-page test note:

Item	2.1	2.2
Page strips	21,000	11,000
Proofreading sheets (7)	10,000	10,000
Output Markdown	6,000	6,000
Text layer read in full	up to 3,200	0
Second copy of the output when assembling	up to 6,000	0
Zooms, `SKILL.md`, manifest, script output	5,000	4,000
Total	˜45,000–51,000	˜31,000

Image tokens use Anthropic's published approximation of width × height/750 after downscaling to a 1,568 px long edge; treat them as estimates.

5 Test results

Two documents with their LaTeX sources were available as ground truth: a five-page note on Eq. (40) of Kugler, Lee & von Delft ("Kugler note") and a ten-page note on particle–vortex duality ("duality note"). Both are single-column and born-digital (pdfTEX). All transcriptions were made from the images before the source was opened. Figure extraction was tested on a third document without source: Lihm, Kiese, Lee & Kugler, PNAS **123**, e2525308123 (2026), eight pages in two columns ("PNAS paper").

Accuracy on the Kugler note

Measure	Whole document	Page 3
Displayed equations exactly correct	25 / 26	8 / 8
Inline formulas exactly correct	143 / 147	38 / 38
Prose	identical	162 / 162 words
`check.py` (KaTeX, tags, debris)	pass	pass

The five whole-document mismatches render identically to the source: `F_{\rm rest}` versus `F_{\mathrm{rest}}`, the arrow written as the Unicode character U+21D2 instead of `$\Rightarrow$`, parentheses outside rather than inside `$...$`, and "K-part" written as text. With the 2.2 pipeline (per-page files, `assemble.py`, `check.py`, `verify_eqs.py`) the assembled document was identical to the original transcription and scored the same.

Blind legibility test (basis for 170 dpi)

Sixteen items, each with two or three near-identical variants (for example ω' versus ω, a bar over $j+1$ versus over j alone, $\mathcal{K}$ versus K, $\tilde{\mathrm{K}}$ versus $\hat{\mathrm{K}}$, $+$ versus $-$ in an exponent, swapped indices) were typeset in the note's fonts. A script drew one variant per item at random for each resolution and kept the key hidden until all readings were recorded.

Effective resolution (dpi)	232	200	170	145	120	100
Image tokens relative to 232 dpi	100%	74%	54%	39%	27%	19%
Items read correctly	16/16	16/16	16/16	16/16	16/16	16/16

The test is forced-choice (the candidate variants were known), which is easier than open transcription; at 120 and 100 dpi about five readings felt uncertain, at 145 dpi and above none did except one overbar extent. 170 dpi was chosen to keep a wide margin.

Open blind transcription at 170 dpi

Pages 4 and 6 of the duality note, chosen as the two pages with the most math-font characters, were transcribed from 170 dpi strips without zooms. Their hazards include tildes on ϕ and $f_{\mu\nu}$, daggers, integral limits $\int_x^\infty$ versus $\int_\infty^x$, signs in exponents and a primed n'.

Page	Displayed equations	Inline formulas	Prose
4	4/4	34/34	identical
6	6/6	36/36	identical

All matches are strict (no `--lenient`). Caveat: the prose of these pages was already known from the text layer; only the mathematics was unseen.

Proofreading test

Errors of random type were planted, with a hidden key, in random equations of a copy of the Kugler transcription; the sheets from `verify_eqs.py` were then reviewed cold.

Round	Planted	Caught on sheets	Caught by compile/KaTeX	Missed
1	1	1	–	0
2	8	5	3	0

Errors caught on the sheets were a dropped prime, three dropped bolds, a dropped bar and $\mathcal{K}$ turned roman; there were no false alarms. Three planted errors produced LaTeX that does not compile, so their equations never reached the sheets; the verifier reported them, but only in its text output, which led to the new instruction in step 5. Limits: nine errors in total, no sign flips or index swaps among them, and compile failures are partly an artifact of how the errors were planted.

Scorer validation. Four realistic errors planted in the page-3 transcription were all flagged at the right places, while spelling-only variants still scored 8/8 and 38/38. With the errors present the mean token similarity stayed at 0.993 and 0.999, which is why only exact counts are meaningful.

Not yet tested against Marker. A page-3 comparison kit is provided separately (`page3_comparison_kit.zip`); Marker's model weights are hosted on `models.datalab.to` and `huggingface.co`, which Claude's sandbox cannot reach, so the kit must be run locally.

Figure extraction

Measure	PNAS paper
Figures found (captions "Fig. 1"–"Fig. 7")	7 / 7
Crops complete, no running text inside	7 / 7
Decorative graphics wrongly taken as figures	0
Embedded images byte-identical to the rendered files	7 / 7
Caption formulas passing KaTeX	48 / 48
Size of the self-contained file	835 KB

The seven figures sit in both columns of the page layout and include vector diagrams, plots and embedded heatmaps. Crops were checked on the check images and, for the one doubtful edge (the frame of panel C in Fig. 7), on the full image. Captions were transcribed from caption crops; one bar was resolved by zooming ($\lambda_{\overline{\mathrm{M}}}$ with a bar, the following $\tilde{S}_{\mathrm{M}}$ without). The figure-free Kugler note assembled byte-identically to version 2.2, and `check.py` still passed. Extraction, crop checks and caption reading for all seven figures cost about 2,500 tokens. The caption accuracy was not scored, since the paper's source was not available.

6 Changes considered and not made

- **Selective proofreading (rejected).** Skipping "simple" equations would save almost nothing: a classifier skipping only equations without bars, primes, bold, typeface-sensitive letters, accents or braces skips 1 of 26 in the Kugler note. Meanwhile the proofreading test found errors in the plainest equations, such as $\mathcal{G}(\mathrm{i}\boldsymbol{\omega}) = 0$.
- **Skipping strips without math fonts (deferred).** No strip of the Kugler note and 4 of 39 strips of the duality note (6% of strip tokens) contain no math-font character; all four were genuinely free of mathematics. 8 of the duality note's 195 inline formulas are set entirely in text fonts (`+`, `+1`, `=`) and are invisible to the detector; here none fell in a skippable strip. The change needs scan detection and hyphen repair of text-layer prose first, and the benefit is small.

7 Known limitations

- **Scanned PDFs (serious).** A scan has no fonts, so `prepare.py` reports every page as `text-ok`. Until fixed, scanned pages must be transcribed from images regardless of what the printout says.
- **Two-column layouts.** Strips run across both columns, so reading order between strips is not guaranteed; equation numbers are detected only in the right quarter of the page, so left-column equations are missed by the tag checklist; proofreading crops span the full page width and include the other column. Figure extraction is not affected, since each caption defines its own column. A full two-column paper has therefore not been converted yet.
- **Figures.** A figure is found only if its caption lies below it and starts with "Fig.", "Figure" or "FIG."; captions above the figure, other labels such as "Scheme 1", figures split across columns or pages, and tables set as graphics need a manual crop. Text inside figures is part of the image and not searchable. Embedded images display in most desktop Markdown editors and previews; some web renderers strip data URIs.
- **Dotted equation numbers.** Tags such as (5.40) are not recognised. `check.py` then reports a correctly transcribed `\tag{5.40}` as "not printed in PDF" and fails.
- **Proofreading sheets.** Equations that do not compile are left off the sheets rather than shown as placeholders; adjacent equations of one `align` environment are merged into one crop; large symbols can be clipped at the crop edge.
- **Fonts.** Math detection matches known math font names. Documents from Word, with unusual fonts, or with bitmap (Type 3) math fonts, as in older arXiv PDFs, may be misclassified.
- **Self-proofreading.** Without a source, the side-by-side check is done by the same model that made the transcription. Sheets cover numbered displays only; unnumbered displays and inline math rely on the syntax check and on re-reading the strips.
- **Context budget.** A 170 dpi strip costs about 600 tokens and a proofreading sheet about 1,500. A ten-page note fits in one conversation; long papers and books should be converted one section or chapter at a time.
- **Narrow evidence.** Accuracy results come from seven transcribed pages of two documents of the same kind; figure extraction from one paper.

8 Installation and requirements

Upload `pdf-to-markdown.zip` in Claude under *Customize* → *Skills* and make sure code execution is enabled. If an earlier version is installed, replace it. The scripts need PyMuPDF and Pillow; `verify_eqs.py` needs pdflatex and `check.py` needs Node.js, and `check.py` installs KaTeX itself if it is missing. All of these were present in Claude's sandbox during testing (PyMuPDF 1.28.2).

9 Planned next

Support two-column layouts by detecting columns and cutting strips, finding equation numbers and cropping proofreading panels per column, then convert a full two-column paper; detect figures whose caption is above them or labelled differently; accept dotted equation numbers; show uncompilable equations as placeholder panels and crop `align` lines and large symbols correctly; treat every page without a text layer as image-only and test on a scanned textbook page; run the Marker comparison locally.